

Equivalence Partitioning, Boundary-Value Analysis, and Negative Testing

SE 413/513 — Software Testing

SE 413/513 — Software Testing
Grand Valley State University

About This Lecture

- Today: given that we can never test every possible input, how do we choose which ones to test? **Equivalence partitioning**, **boundary-value analysis**, and **negative testing** are the three core answers.
- Builds directly on *Testing Foundations* (Principles 1, 2, and 6) and the *pytest* reading — today's worked examples reuse `calculate_late_fee()` and end by encoding the design as a parametrized *pytest* suite.
- First exposure to these techniques by name, even though you've already informally seen the bug they're designed to catch.

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions, contact hasaka@gvsu.edu. Please cite this material when reproducing it.

Learning Objectives

By the end of today, you should be able to:

- Explain why exhaustive testing is infeasible, and how equivalence partitioning responds to that constraint.
- Identify valid and invalid equivalence classes for a given input specification.
- Apply boundary-value analysis to target the exact inputs where off-by-one defects are most likely to hide.
- Design negative test cases that check for graceful, well-defined failure rather than crashes or silently wrong output.
- Combine equivalence partitioning, boundary-value analysis, and negative testing into one parametrized pytest suite.

Roadmap

- 1 Why We Need Systematic Test-Case Design
- 2 Equivalence Partitioning
- 3 Boundary-Value Analysis
- 4 Negative Testing
- 5 Putting It Together: A Worked Example
- 6 How Much Is Enough?
- 7 Recap

The Problem: We Can't Test Everything

Recall Principle 2 from *Testing Foundations*: **exhaustive testing is impossible**. A function with one 32-bit integer parameter already has over four billion possible inputs; two such parameters, over 1.8×10^{19} .

This raises a practical question: **if we cannot test every input, which inputs *should* we test?**

Picking inputs at random, or “some easy-looking numbers,” tends to produce suites that are both **bloated** (redundant cases exercising the same code path) and **full of holes** (whole categories of input, especially unusual or invalid ones, never tested).

Black-Box vs. White-Box: Where These Techniques Fit

Equivalence partitioning and **boundary-value analysis** are the two most fundamental techniques for choosing test inputs systematically, based on the *specification* rather than the code. Together with **negative testing** — deliberately testing invalid input — they are today's core toolkit.

Key Idea

These are **black-box** techniques: derived entirely from what the software is *supposed* to do, not from reading its source code. Contrast **white-box** techniques (statement or branch coverage), derived from the code's internal structure. Black-box design is what you use *before* code even exists; white-box coverage is what you use *afterward*, to check whether your black-box tests actually exercised the implementation.

Roadmap

- 1 Why We Need Systematic Test-Case Design
- 2 Equivalence Partitioning**
- 3 Boundary-Value Analysis
- 4 Negative Testing
- 5 Putting It Together: A Worked Example
- 6 How Much Is Enough?
- 7 Recap

The Basic Idea

Equivalence partitioning (EP) divides the space of possible inputs into a small number of **equivalence classes**, such that the software is expected to behave the *same way* for every input within a given class.

Key Idea

Core assumption: *if the software works correctly for one representative value in a class, it probably works correctly for all values in that class.* Instead of testing every possible input, test **one representative value from each class** — turning an infeasible problem into a manageable one.

Valid and Invalid Classes

Every input specification implicitly (or explicitly) divides inputs into:

- **Valid classes:** inputs the software is specifically designed to accept and process normally.
- **Invalid classes:** inputs outside the specification — out of range, wrong type, malformed — for which the software should still behave in some well-defined way (typically, reject cleanly).

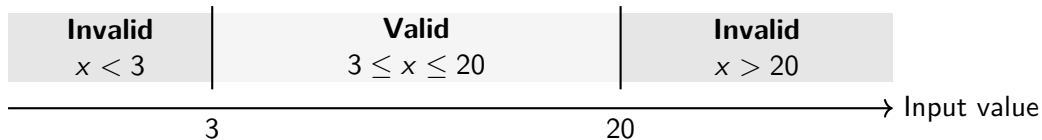
Common Pitfall

It is easy to design test cases only for valid classes and forget invalid ones entirely. Excellent coverage of valid inputs and zero coverage of invalid inputs tells you nothing about how your software behaves when it inevitably receives bad data — and bad data is inevitable. More on this in the Negative Testing section.

Worked Example: Username Validation

Goal: Divide the input space into groups (equivalence classes) where the program is expected to behave similarly.

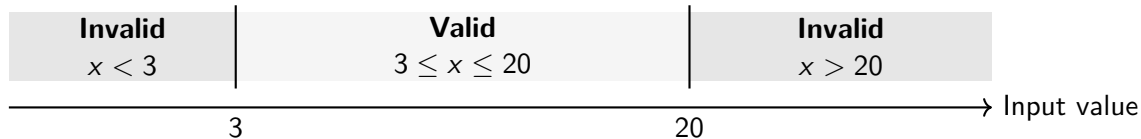
Recall `is_valid_username(username)` from the *pytest* reading's Exercise 1: return `True` for usernames between 3 and 20 characters (inclusive), `False` otherwise.



- Instead of testing every possible input, identify a small number of representative classes.
- Choose at least one test case from each class.
- The key assumption: inputs within the same class should produce similar program behavior.
- EP reduces the number of tests while still providing coverage of different expected behaviors.

Worked Example: Username Validation

True if length is between 3 and 20 characters (inclusive); False otherwise.



Class	Description	Representative test
EC1	Length < 3	"ab" (length 2)
EC2	$3 \leq \text{Length} \leq 20$	"alice" (length 5)
EC3	Length > 20	25-character string

Key idea

One representative test from each equivalence class gives coverage of the expected behaviors—*under the equivalence assumption*.

Multiple Parameters: Classes Multiply

Real functions often take more than one parameter, and each may have its own equivalence classes. Two parameters with 3 classes each means $3 \times 3 = 9$ combinations to test *every combination*; three parameters, 27. Combinations grow multiplicatively.

Tip

In practice, teams typically test each class individually (holding other parameters at a typical valid value), and reserve full combinatorial or pairwise testing for cases where parameter *interactions* are a specific, high-risk concern.

Pairwise Testing, Case 1: Two Parameters (3×3)

Setup. P_1 and P_2 each have 3 equivalence classes (A, B, C).

Full combinatorial: $3 \times 3 = 9$ test cases.

Pairwise coverage: every pair (P_1, P_2) is the thing we're testing when there are only two parameters — so pairwise testing needs the same 9 cases as full combinatorial.

Tip

Key point: pairwise testing only saves effort once there are *three or more* parameters. With two, there's nothing to save.

P_1 class ↑	P_2 class →		
	A	B	C
A	1	2	3
B	4	5	6
C	7	8	9

All **9 of 9** combinations tested — full coverage.

Pairwise Testing, Case 2: Three Parameters ($3 \times 3 \times 3$)

P_1, P_2, P_3 each have 3 classes $\Rightarrow$ full combinatorial = $3 \times 3 \times 3 = 27$ test cases. Pairwise coverage (every pair of values appears together at least once) needs only **9**.

Full combinatorial — 27 tests

$P_3 = A$

$P_3 = B$

$P_3 = C$

Every 3-way combination (P_1, P_2, P_3) is exercised: 27 filled cells.

Pairwise-covering design — 9 tests

#	P_1	P_2	P_3
1	A	A	A
2	A	B	B
3	A	C	C
4	B	A	B
5	B	B	C
6	B	C	A
7	C	A	C
8	C	B	A
9	C	C	B

These 9 rows still cover every pair:

$P_1 \times P_2$
9/9

$P_1 \times P_3$
9/9

$P_2 \times P_3$
9/9

Tip

Why this matters: pairwise testing catches every 2-parameter interaction fault with 9 tests instead of 27 — a reduction that grows sharply with more parameters or classes, at the cost of not exercising every 3-way (or higher) combination.

Roadmap

- 1 Why We Need Systematic Test-Case Design
- 2 Equivalence Partitioning
- 3 Boundary-Value Analysis**
- 4 Negative Testing
- 5 Putting It Together: A Worked Example
- 6 How Much Is Enough?
- 7 Recap

Why Boundaries Matter

Decades of defect data support a consistent observation: **defects cluster at the edges of equivalence classes far more often than in their interior.**

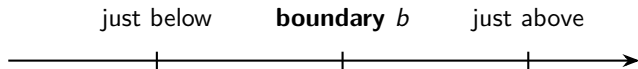
Boundaries are exactly where a programmer is most likely to make an off-by-one mistake: $<$ instead of $<=$, $>$ instead of $>=$, or confusion about whether a range is inclusive or exclusive.

Key Idea

Boundary-value analysis (BVA) deliberately tests *at* and *immediately around* the edges of each class, rather than relying on one arbitrary value from the middle. EP tells you *which classes* exist; BVA tells you *which values* within and around them are most worth testing.

The Technique: Below, At, Above

For each boundary, BVA tests three values: just below it, the boundary itself, and just above it.



For a class specified as $3 \leq \text{length} \leq 20$: the boundary at 3 is tested with 2, 3, 4; the boundary at 20 is tested with 19, 20, 21.

Boundary	Just Below	At	Just Above
Lower (3)	2 (invalid)	3 (valid)	4 (valid)
Upper (20)	19 (valid)	20 (valid)	21 (invalid)

A Boundary Bug in `calculate_late_fee()`

Recall the fault discussed in *Testing Foundations*:

```
def calculate_late_fee(days_late, daily_rate=1.50, max_fee=25.00):  
    fee = days_late * daily_rate  
    if fee > max_fee:  
        return max_fee  
    return fee
```

The condition `fee > max_fee` has a boundary at `fee == max_fee`. BVA means specifically testing the input that makes `fee` equal `max_fee` — not just an arbitrary “large” value.

```
>>> calculate_late_fee(days_late=25/1.50)    # fee == 25.0 exactly  
25.0
```

Here the boundary happens to be handled correctly. But `fee >= max_fee`, or a careless `min()`, would put the mistake *exactly* here — and `days_late=30` would never expose it.

A Habit Worth Building

Tip

Whenever you see a comparison operator ($<$, $<=$, $>$, $>=$, $==$) in a specification or in code you are testing, ask: “*what is the exact value where this condition flips from true to false?*” — and write a test for that exact value.

Real-World Case Study: The Zune 30GB Freeze

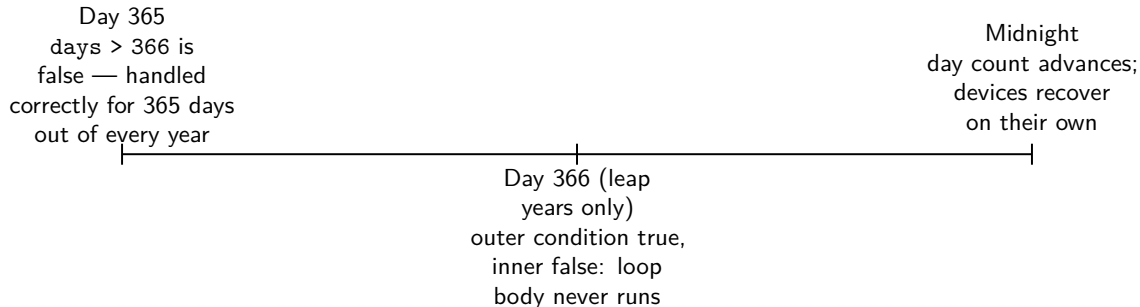
Real-World Case Study

December 31, 2008 — a leap year's last day. Every Microsoft Zune 30GB froze simultaneously. The device driver converted an internal day-count into a year using a loop like:

```
while days > 365:
    if is_leap_year(year):
        if days > 366:
            days -= 366; year += 1
    else:
        days -= 365; year += 1
```

On day 366 of a leap year, the outer condition (`days > 365`) was true, but the inner condition (`days > 366`) was false — so nothing executed, `days` never changed, and the loop ran forever.

The Zune Timeline



Key Idea

The code correctly handled `days > 366`, but not `days == 366` — a textbook boundary-value defect, invisible for all but one specific day.

Why This Specific Date?

Discussion Prompt

The Zune bug depended on the *specific date* (day 366 of a leap year) rather than being present at all times. Why? What does that tell you about how long a boundary-value defect like this can hide before it surfaces?

Roadmap

- 1 Why We Need Systematic Test-Case Design
- 2 Equivalence Partitioning
- 3 Boundary-Value Analysis
- 4 Negative Testing**
- 5 Putting It Together: A Worked Example
- 6 How Much Is Enough?
- 7 Recap

Positive vs. Negative Tests

A **positive test** checks that the software does the right thing with *valid* input — the “happy path.” A **negative test** checks that it does the right thing with *invalid* input: out of range, wrong type, missing, malformed, or otherwise unexpected.

Key Idea

“The right thing” for invalid input is almost never “crash with a confusing, unhandled error,” and it is even worse to silently produce a wrong-but-plausible result. The right thing is usually a well-defined, well-documented failure: a clear exception with an informative message, a validation error, or a logged rejection.

What Counts as Invalid Input?

- **Out-of-range values:** a negative age, a percentage above 100, a date far in the future.
- **Wrong type:** a string where a number is expected, or `None` where an object is expected.
- **Missing or empty data:** an empty string, an empty list, a missing required field.
- **Malformed data:** an email with no @, a badly-formatted date, invalid JSON.
- **Extreme or adversarial values:** an extremely long string, unusual Unicode, unexpected whitespace.

Invalid equivalence classes (out-of-range values) are one *source* of negative tests — but negative testing is broader, including invalid values (like wrong types) that don't fit neatly into the same numeric classes at all.

Negative Testing Exposes What EP Alone Misses

The original equivalence classes for `calculate_late_fee()`'s `days_late` were implicitly “typical small numbers” and “numbers large enough to trigger the cap.” Nobody defined an invalid class for negative values — because the requirement never said what should happen:

```
>>> calculate_late_fee(-4)
-6.0
```

Key Idea

A systematic negative-testing pass asks, for *every* parameter, regardless of what the spec says: *what happens if this is negative? Zero? Not a number at all?* This is exactly the discipline that would have caught this fault before release.

Real-World Case Study: The Year 2000 Problem

Real-World Case Study

For decades, many systems stored calendar years using only two digits ("99" instead of "1999") to save memory. This implicitly treated “two-digit year” as the *only* class worth handling — nobody systematically asked what should happen at the rollover from "99" to "00".

When date arithmetic assumed the year always increased, a rollover to "00" could be read as 1900 instead of 2000, corrupting ages, sort order, and interest calculations. Remediation ahead of January 1, 2000 became one of the largest coordinated software efforts in history (global estimates commonly cited in the \$300–600 billion range).

Key Idea

Y2K is a boundary-value story at civilization scale: a foreseeable boundary, expensive precisely because it was not designed for early — and an unexamined equivalence class (“years always increase”), not a single line-of-code bug.

Roadmap

- 1 Why We Need Systematic Test-Case Design
- 2 Equivalence Partitioning
- 3 Boundary-Value Analysis
- 4 Negative Testing
- 5 Putting It Together: A Worked Example**
- 6 How Much Is Enough?
- 7 Recap

The Spec and a Plausible Implementation [Let's Try]

Given a package weight in kg, return the shipping cost: \$5.00 for $0 < wt \leq 1$, \$10.00 for $1 < wt \leq 5$, \$20.00 for $5 < wt \leq 20$. Weight ≤ 0 or > 20 is invalid: raise `ValueError`.

```
def determine_shipping_cost(weight_kg):  
    if weight_kg <= 0:  
        raise ValueError("weight_kg must be positive")  
    if weight_kg > 20:  
        raise ValueError("packages over 20kg require freight shipping")  
    if weight_kg <= 1:  
        return 5.00  
    elif weight_kg < 5:  
        return 10.00  
    else:  
        return 20.00
```

The Spec and a Plausible Implementation

Given a package weight in kg, return the shipping cost: \$5.00 for $0 < wt \leq 1$, \$10.00 for $1 < wt \leq 5$, \$20.00 for $5 < wt \leq 20$. Weight ≤ 0 or > 20 is invalid: raise `ValueError`.

```
def determine_shipping_cost(weight_kg):  
    if weight_kg <= 0:  
        raise ValueError("weight_kg must be positive")  
    if weight_kg > 20:  
        raise ValueError("packages over 20kg require freight shipping")  
    if weight_kg <= 1:  
        return 5.00  
    elif weight_kg < 5:      # <-- suspicious: compare to the spec above  
        return 10.00  
    else:  
        return 20.00
```

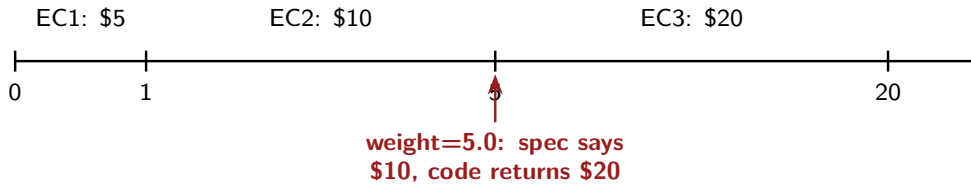
Step 1: Equivalence Classes

Class	Description	Valid?	Example
EC1	$0 < \text{wt} \leq 1$	Valid	0.5
EC2	$1 < \text{wt} \leq 5$	Valid	3.0
EC3	$5 < \text{wt} \leq 20$	Valid	10.0
EC4	$\text{wt} \leq 0$	Invalid	-1.0
EC5	$\text{wt} > 20$	Invalid	25.0

Five classes; the interior representatives above, tested alone, would never catch what's coming next.

Step 2: Boundary Values — and a Bug

Four boundaries: 0, 1, 5, and 20.



Per the spec, $wt = 5.0$ belongs to EC2 ($1 < wt \leq 5$) and should cost \$10.00. But `elif weight_kg < 5` is **strict**, so 5.0 falls through to `else` and incorrectly returns \$20.00.

Key Idea

Exactly the class of fault BVA exists to catch — and testing only an interior value like 3.0 for EC2 would never reveal it.

Step 3: Negative Test Cases

Beyond the invalid numeric ranges already in EC4/EC5, a thorough pass also asks about invalid *types*:

```
>>> determine_shipping_cost(None)
Traceback (most recent call last):
...
TypeError: '<=' not supported between instances of
'NoneType' and 'int'
```

The function does fail — but with a confusing, implementation-level message that leaks Python internals, instead of a clear, documented error. A real (if minor) defect that only a deliberate “wrong type entirely” test would surface.

Step 4a: Encode Valid Weights — Including the Bug

The three techniques combine into one parametrized pytest suite, exactly as in the *pytest* reading:

```
@pytest.mark.parametrize("weight_kg, expected_cost", [
    (0.01, 5.00), (0.5, 5.00), (1.0, 5.00),          # EC1
    (1.01, 10.00), (3.0, 10.00),                    # EC2 interior
    pytest.param(5.0, 10.00,                          # EC2 boundary
        marks=pytest.mark.xfail(reason="boundary bug: < should be <=")),
    (5.01, 20.00), (10.0, 20.00), (20.0, 20.00),    # EC3
])
def test_valid_weights(weight_kg, expected_cost):
    assert determine_shipping_cost(weight_kg) == expected_cost
```

The known boundary bug is reported as XFAIL, not silently passing or breaking the build.

Step 4b: Encode the Negative Tests

```
# Negative testing: invalid numeric ranges (EC4, EC5) and the  
# boundaries immediately outside the valid region.  
@pytest.mark.parametrize("weight_kg", [-1.0, -0.01, 0.0, 20.01, 25.0])  
def test_invalid_weight_raises_value_error(weight_kg):  
    with pytest.raises(ValueError):  
        determine_shipping_cost(weight_kg)  
  
# Negative testing: invalid types entirely.  
@pytest.mark.parametrize("bad_input", [None, "5", [5], {}])  
def test_invalid_type_raises_a_clear_error(bad_input):  
    with pytest.raises((TypeError, ValueError)):  
        determine_shipping_cost(bad_input)
```

Three equivalence classes, four boundaries, and two flavors of negative input — eighteen focused test cases, not an arbitrary, unsystematic handful.

Roadmap

- 1 Why We Need Systematic Test-Case Design
- 2 Equivalence Partitioning
- 3 Boundary-Value Analysis
- 4 Negative Testing
- 5 Putting It Together: A Worked Example
- 6 How Much Is Enough?**
- 7 Recap

How Much Is Enough?

EP and BVA do not mean testing every boundary of every parameter of every function with equal intensity. Recall Principle 6 from *Testing Foundations*: **testing is context-dependent**.

Tip

Invest the most in EP/BVA for code that (a) implements business rules with explicit thresholds, (b) handles money, safety, or security, or (c) has a history of off-by-one defects — exactly where a boundary mistake is both likely and costly.

Common Pitfalls

Common Pitfall

- **Testing only the “obvious middle.”** A value like 50 as “a representative percentage” feels natural but is the least likely value to expose an off-by-one defect. Always pair EP with BVA.
- **Defining invalid classes only for what the spec explicitly forbids.** A spec rarely spells out negative ages or None — but a real caller will eventually pass them. Ask about every parameter.
- **Confusing “didn’t crash” with “handled correctly.”** A negative test should assert on a *specific* exception or error, not merely that the program kept running.

Roadmap

- 1 Why We Need Systematic Test-Case Design
- 2 Equivalence Partitioning
- 3 Boundary-Value Analysis
- 4 Negative Testing
- 5 Putting It Together: A Worked Example
- 6 How Much Is Enough?
- 7 Recap**

A Practical Mental Model

When you see a spec like **“accepts values from 1 to 100,”** think: *“Three classes: below 1, from 1 to 100, above 100. Six boundary values: 0, 1, 2 and 99, 100, 101.”*

When you see a **comparison operator** in code, think: *“What is the exact input that makes this condition flip? That is my next test case.”*

When you finish writing **“happy path”** tests, think: *“What is every way a caller could hand this function garbage — and does it fail clearly when they do?”*

Key Takeaways

- **Systematic beats exhaustive.** EP and BVA exist because exhaustive testing is infeasible, not merely inconvenient.
- **Partition first, then target the edges.** EP identifies classes; BVA chooses the highest-value inputs within and around them.
- **Every parameter has invalid classes**, whether the spec says so or not.
- **A comparison operator is a promise to test.** Every $<$, $<=$, $>$, $>=$ names an exact input worth a dedicated test.
- **Negative testing checks for graceful failure**, not just survival.
- **These techniques are context-dependent** — invest more where money, safety, security, or business-critical thresholds are involved.

One Last Question — and What's Next

Discussion Prompt

A teammate says: “We don’t need boundary-value analysis if we already have equivalence partitioning — one value per class is enough.” Using today’s shipping-cost example, argue why this is wrong.

Next time: Coverage-Based Testing — the *white-box* counterpart to today’s black-box techniques. Once you’ve designed tests from the specification, how do you check whether they actually exercised the implementation?

Questions?

Thank you.

SE 413/513 — Software Testing
Grand Valley State University